
Entity Framework



Table des matières

1. Entity Framework, c'est quoi?
 - a. Principe d'ORM
 - b. ORM dans les autres langages
 - c. Historique d'Entity Framework
2. Aborder Entity Framework
3. Installation d'Entity Framework
 - a. Modification de la configuration
4. Espace de noms
5. Database First
 - a. Connexion à la base de données
 - b. Modification de la structure
6. Code First
 - a. to New Database
 - i. Création de la "connectionString"
 - ii. Création des modèles
 - iii. Création du DbContext

- iv. Relation One to Many
- v. Relation One to One
- vi. Relation Many to Many
- vii. Data Annotations
- viii. Héritage
 - a. Table per Hierarchy (TPH)
 - b. Table per Type (TPT)
 - c. Table per Concrete Class (TPC)
- ix. Migration
- x. Exercice
- b. to Existing Database

Qu'est ce qu'Entity Framework ?



Entity Framework est un ORM (object-relational mapping) open source créé par Microsoft.

L'objectif principal d'Entity Framework est de fournir une séparation entre la couche de données et la gestion des données afin de faciliter le travail du développeur.

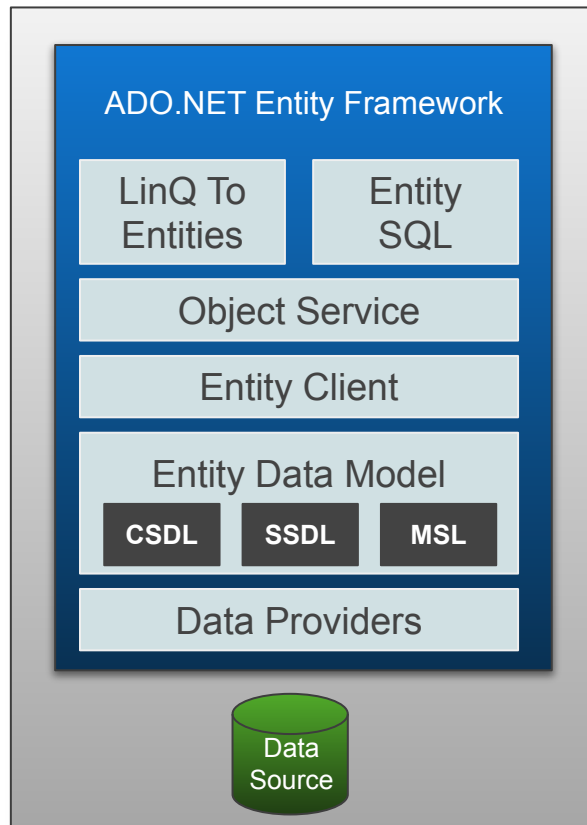
Principe d'Object Relational Model

Un ORM est considéré comme une technique de programmation qui permet de transformer (mapper) automatiquement la base de données relationnelle vers des objets utilisables en programmation orientée objet (ou inversement).

L'ORM s'appuie sur un fichier de configuration associant les classes de l'application au schéma de la base de données.

Ci après, un schéma représentant l'architecture générale d'Entity Framework.

Principe d'Object Relational Model



Data Source : Il s'agit de la première couche de données qui peut être stockée dans une ou plusieurs DB.

Data Providers : L'accès aux données se fait par un fournisseur d'accès aux données ADO .NET (SQL Server, MySQL, Oracle, etc.)

Entity Data Model : l'EDM contient 3 sections distinctes :

- Conceptual Schema Definition Language (CSDL) : Déclare et définit les entités, associations et l'héritage. Les entités sont générées à partir de ce schéma.
- Store Schema Definition Language (SSDL) : Métadonnées décrivant le conteneur de stockage (= DB) qui contient les données
- Mapping Specification Language (MSL) : Mappe les entités du fichier CSDL aux tables décrites dans le fichier SSDL.

Entity Client : Entity Client est un fournisseur ADO .NET qui supporte l'accès aux données décrit dans l'EDM. C'est similaire à un fournisseur classique tel que SQLClient ou OracleClient. Il fournit divers composants tel que EntityCommand, EntityConnection et EntityTransaction.

Object Service : Ce composant permet d'interroger les objets (CRUD) en utilisant soit SQL Entities ou LinQ to Entities.

LinQ To Entities : Permet l'interrogation d'entité définie dans l'EDM.

Entity SQL : Entity SQL est un dérivé de Transact-SQL, conçu pour interroger et manipuler des entités définies dans l'EDM. Il prend en charge l'héritage et les associations. Il peut être utilisé aussi bien sur les composants Object Services et Entity Client.

ORM dans les autres langages

Voici une liste non exhaustive d'ORM dans d'autres langages.

Les plus connus sont sans doute :

1. Hibernate et JPA pour Java
2. NHibernate (et Entity Framework) pour .NET
3. Doctrine et CakePHP pour PHP

Historique d'Entity Framework

La première version d'Entity Framework a été fournie avec le Framework .NET 3.5 SP1 et Visual Studio 2008 sorti en août 2008.

La seconde version a été nommée Entity Framework 4.0 et mis en service avec le Framework .NET 4.0 en 2010.

Le support de Code First est apparu avec la version 4.1 en juillet 2011.

En Février 2012 est sorti la version 4.3.1 et apportait une nouveauté : le support des migrations.

La version 5 sortie en août 2012 visait la version 4.5 du framework .NET.

La version 6, que nous utilisons ici est sortie en octobre 2013 et est maintenant Open Source. Les fichiers sources sont disponible sur GitHub.

Aborder Entity Framework

Entity Framework propose trois types d'approches différents :
Model First, Database-First et Code-First. Nous n'aborderons ici que les deux dernières approches.
Le plus répandu est très clairement Code-First

Database-First :



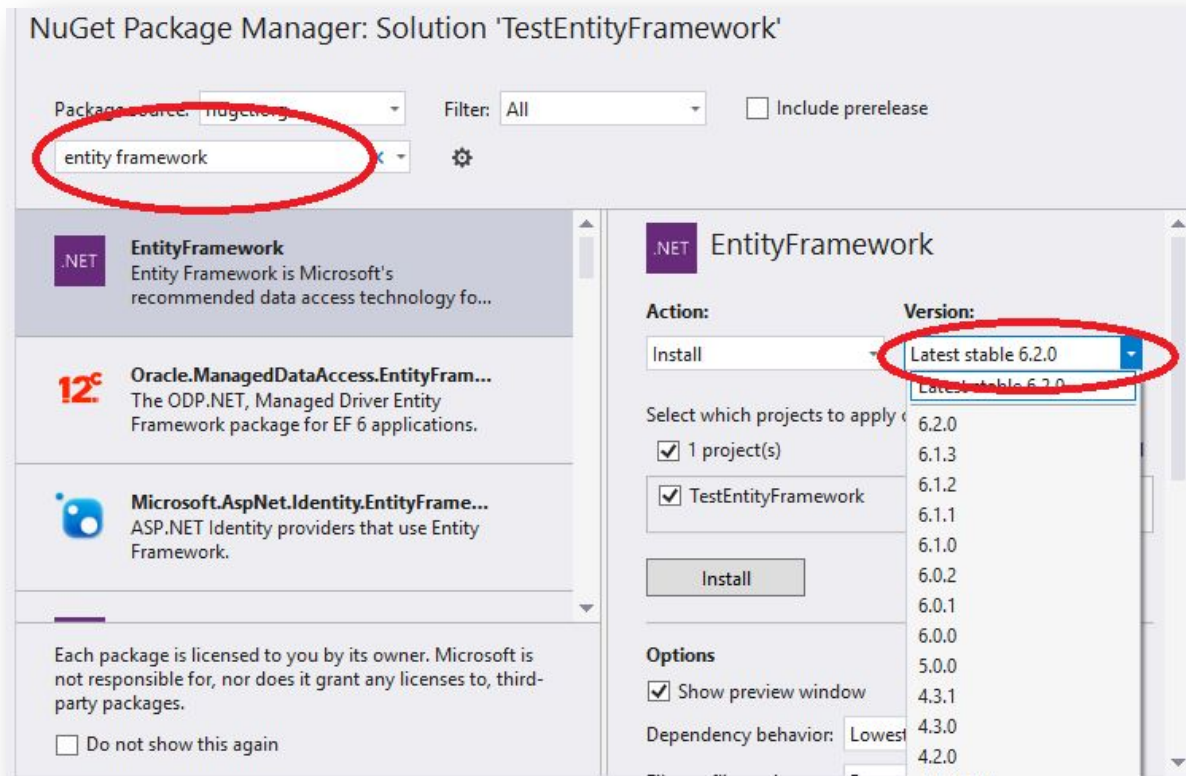
Vous avez déjà une base de données existantes, vous pouvez utiliser cette approche afin de créer vos objets POCO.

Code-First :



Vous définissez vous même vos modèles qui seront transformés en une base de données
ou
vous pouvez aussi vous connecter à une base de données existante.

Installation d'Entity Framework



Pour pouvoir utiliser Entity Framework,, vous devez procéder à son installation via Nuget Package.

Accédez au menu “Tools” -> Nuget Package Manager -> Manage Nuget Package for Solution

Recherchez le terme : “Entity Framework”.

Vous pouvez remarquer que vous pouvez choisir la version à installer. Par défaut, c’est la dernière version stable qui est sélectionnée.

Pour procéder à l’installation, cliquez sur le bouton “Install”.



Le package doit être installé individuellement pour chaque solution!

Modification de la configuration

Ouvrez maintenant le fichier app.config. Par défaut lors de la création du projet, voici ce que contient ce fichier

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <startup>
    <supportedRuntime version="v4.0" sku=".NETFramework,Version=v4.5.2" />
  </startup>
</configuration>
```

On remarque aisément que le fichier a été fortement modifié. Des informations à propos d'Entity Framework ont été ajoutées ainsi que les informations concernant le provider par défaut.

Si vous travaillez en Code-First, vous devrez ajouter la "connectionString" dans ce fichier pour l'accès à la DB. Cela se fait automatiquement en Database-First..

Entity Data Model - Espace de noms

Tout comme pour les espaces de noms que l'on rencontre dans nos classes, un espace de noms dans le modèle EDM permet aux entités de même nom d'être différenciée afin d'éviter une ambiguïté au niveau du compilateur.

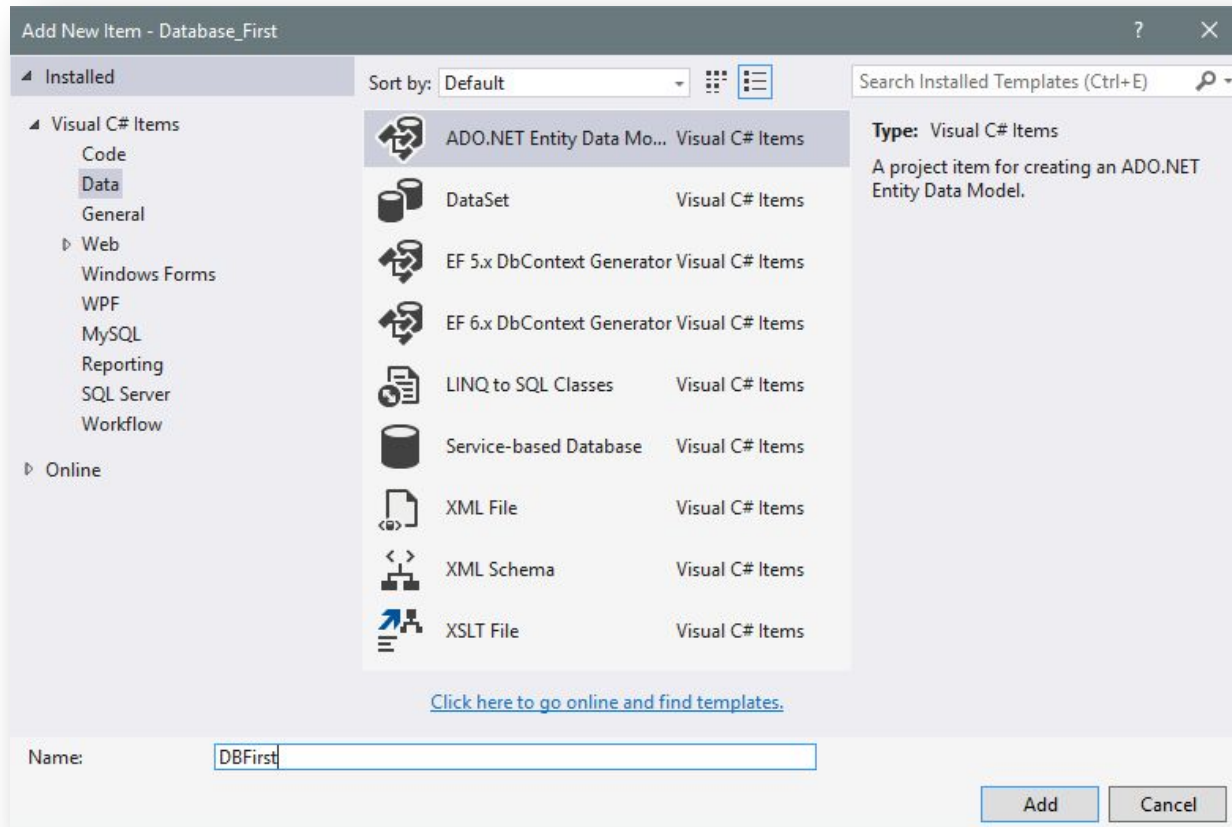
Database First

Idéal lorsque la base de données est existante

Permet de ne pas s'occuper de la gestion de la base de données

Connexion à la base de données très facile avec l'assistant EDM (**E**ntity **D**ata **M**odel) de Visual Studio

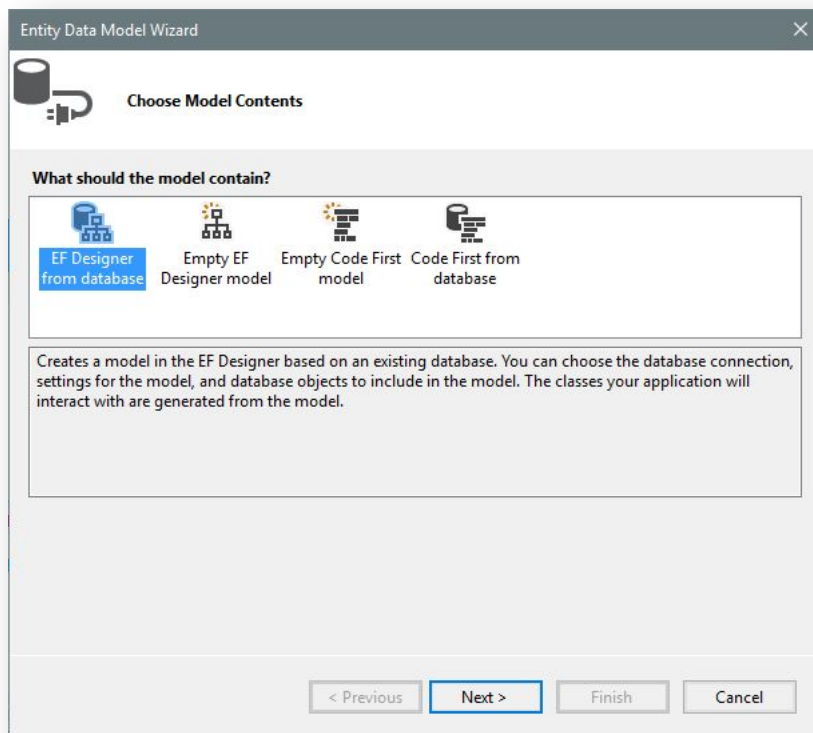
Database First : Connexion à la base de données



Appuyez sur CTRL + Shift + A pour ajouter un nouvel élément.

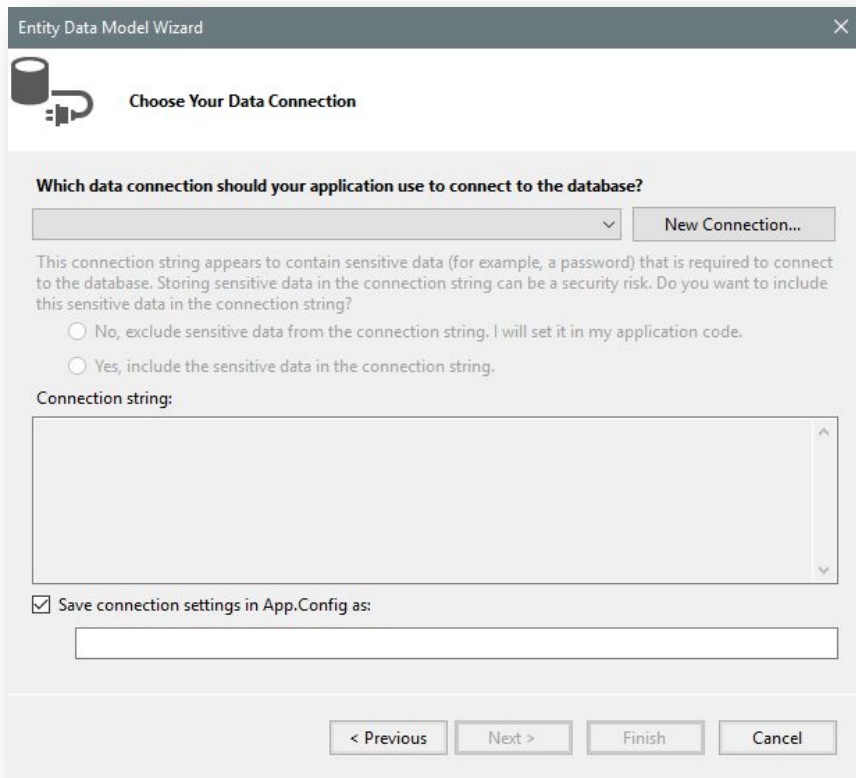
Sélectionnez Data puis ADO.NET Entity Data Model. Donnez un nom puis cliquez ensuite sur Add

Database First : Connexion à la base de données



Sélectionnez EF Designer from Database puis cliquez sur “Next”.

Database First : Connexion à la base de données



The image shows a screenshot of the 'Entity Data Model Wizard' dialog box, specifically the 'Choose Your Data Connection' step. The dialog has a title bar with the text 'Entity Data Model Wizard' and a close button. Below the title bar is a header area with a database icon and the text 'Choose Your Data Connection'. The main content area contains the question 'Which data connection should your application use to connect to the database?'. Below this question is a dropdown menu and a 'New Connection...' button. A text block explains that the connection string might contain sensitive data like a password and asks if it should be included. There are two radio buttons: 'No, exclude sensitive data from the connection string. I will set it in my application code.' and 'Yes, include the sensitive data in the connection string.'. Below the radio buttons is a 'Connection string:' label and a large text area for entering the connection string. At the bottom left, there is a checkbox labeled 'Save connection settings in App.Config as:' followed by a text field. At the bottom right, there are four buttons: '< Previous', 'Next >', 'Finish', and 'Cancel'.

Entity Data Model Wizard

Choose Your Data Connection

Which data connection should your application use to connect to the database?

This connection string appears to contain sensitive data (for example, a password) that is required to connect to the database. Storing sensitive data in the connection string can be a security risk. Do you want to include this sensitive data in the connection string?

☐ No, exclude sensitive data from the connection string. I will set it in my application code.

☐ Yes, include the sensitive data in the connection string.

Connection string:

☒ Save connection settings in App.Config as:

< Previous Next > Finish Cancel

Aucune connexion n'ayant jamais été établie, il nous faut nous connecter à notre base de données.

Cliquez sur "New Connection".

Database First : Connexion à la base de données

Connection Properties

Enter information to connect to the selected data source or click "Change" to choose a different data source and/or provider.

Data source:
Microsoft SQL Server (SqlClient) Change...

Server name:
BS-TEN-DELL01\SQL Refresh

Log on to the server

☒ Use Windows Authentication

☐ Use SQL Server Authentication

User name:

Password:

☐ Save my password

Connect to a database

☒ Select or enter a database name:
Database_First

☐ Attach a database file:
 Browse...

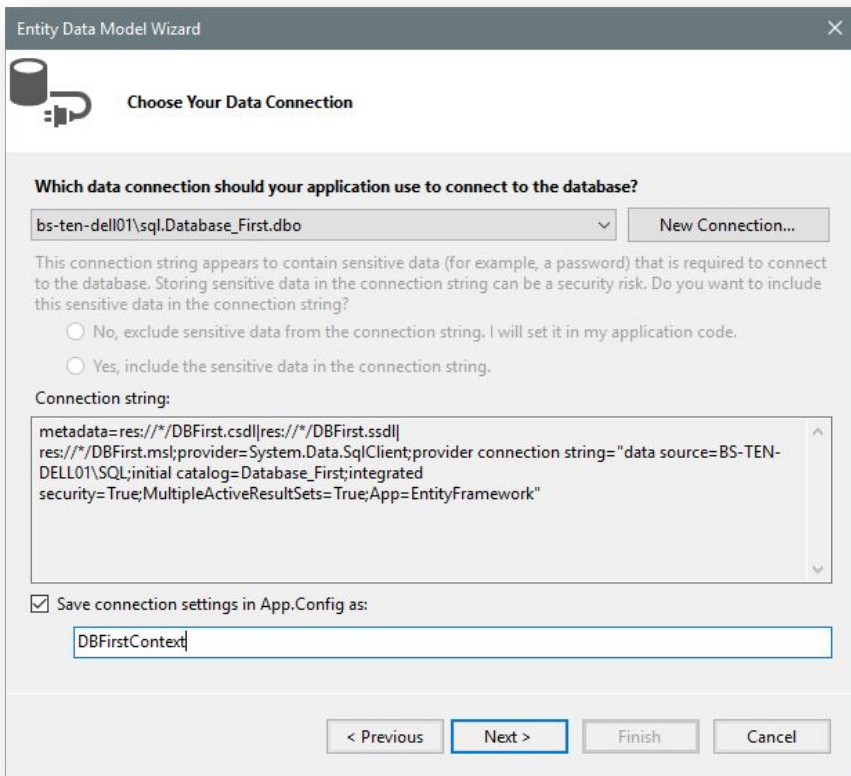
Logical name:

Advanced...

Test Connection OK Cancel

Dans "Server Name", indiquez le nom de votre serveur et l'instance à utiliser. Sélectionnez la base de données à utiliser puis cliquez sur "OK"

Database First : Connexion à la base de données



The screenshot shows the 'Entity Data Model Wizard' window, specifically the 'Choose Your Data Connection' step. The window has a title bar with the text 'Entity Data Model Wizard' and a close button. Below the title bar is a header area with a database icon and the text 'Choose Your Data Connection'. The main content area contains the question 'Which data connection should your application use to connect to the database?'. Below this is a dropdown menu showing 'bs-ten-dell01\sql.Database_First.dbo' and a 'New Connection...' button. A warning message states: 'This connection string appears to contain sensitive data (for example, a password) that is required to connect to the database. Storing sensitive data in the connection string can be a security risk. Do you want to include this sensitive data in the connection string?'. There are two radio buttons: 'No, exclude sensitive data from the connection string. I will set it in my application code.' (selected) and 'Yes, include the sensitive data in the connection string.'. Below this is a 'Connection string:' label and a text box containing the connection string: 'metadata=res://*/DBFirst.csdl|res://*/DBFirst.ssdl|res://*/DBFirst.msl;provider=System.Data.SqlClient;provider connection string="data source=BS-TEN-DELL01\SQL;initial catalog=Database_First;integrated security=True;MultipleActiveResultSets=True;App=EntityFramework"'. At the bottom, there is a checkbox 'Save connection settings in App.Config as:' which is checked, and a text box containing 'DBFirstContext'. At the very bottom are four buttons: '< Previous', 'Next >', 'Finish', and 'Cancel'.

Entity Data Model Wizard

Choose Your Data Connection

Which data connection should your application use to connect to the database?

bs-ten-dell01\sql.Database_First.dbo New Connection...

This connection string appears to contain sensitive data (for example, a password) that is required to connect to the database. Storing sensitive data in the connection string can be a security risk. Do you want to include this sensitive data in the connection string?

☐ No, exclude sensitive data from the connection string. I will set it in my application code.

☐ Yes, include the sensitive data in the connection string.

Connection string:

metadata=res://*/DBFirst.csdl|res://*/DBFirst.ssdl|res://*/DBFirst.msl;provider=System.Data.SqlClient;provider connection string="data source=BS-TEN-DELL01\SQL;initial catalog=Database_First;integrated security=True;MultipleActiveResultSets=True;App=EntityFramework"

☒ Save connection settings in App.Config as:

DBFirstContext

< Previous Next > Finish Cancel

La connexion étant maintenant établie, il suffit de cliquer sur "Next"

Database First : Connexion à la base de données

Data source:
Microsoft SQL Server (SqlClient) Change...

Server name:
(local)\sql Refresh

Log on to the server

☒ Use Windows Authentication
☐ Use SQL Server Authentication

User name:
Password:
☐ Save my password

Connect to a database

☒ Select or enter a database name:
EFDatabase

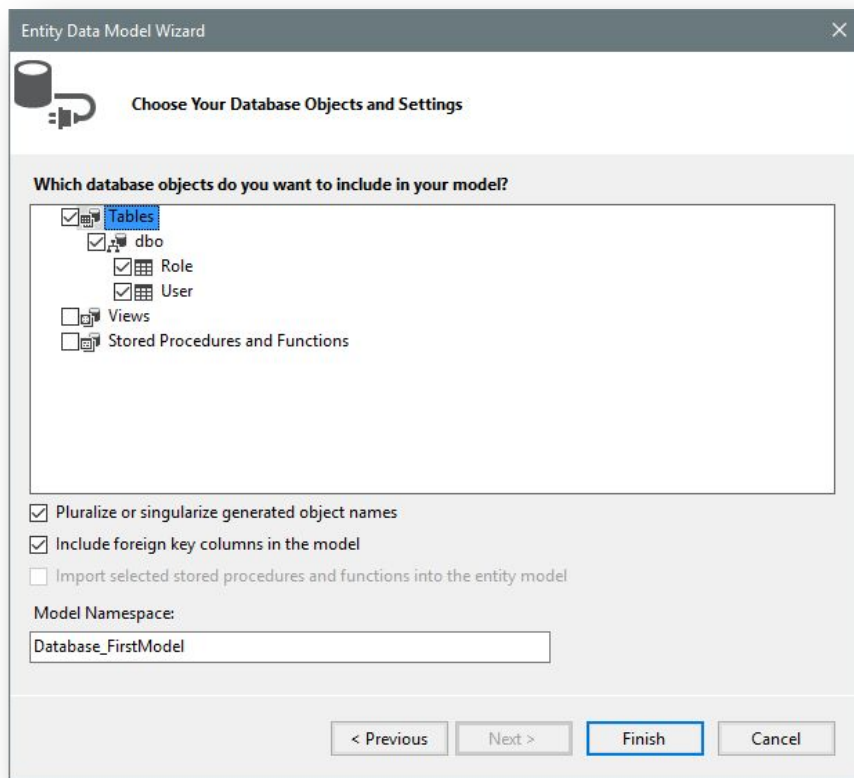
☐ Attach a database file:
 Browse...
Logical name:

Advanced...

Test Connection OK Cancel

Sur cet écran, il est nécessaire de créer une nouvelle connexion vers la base de données existante.

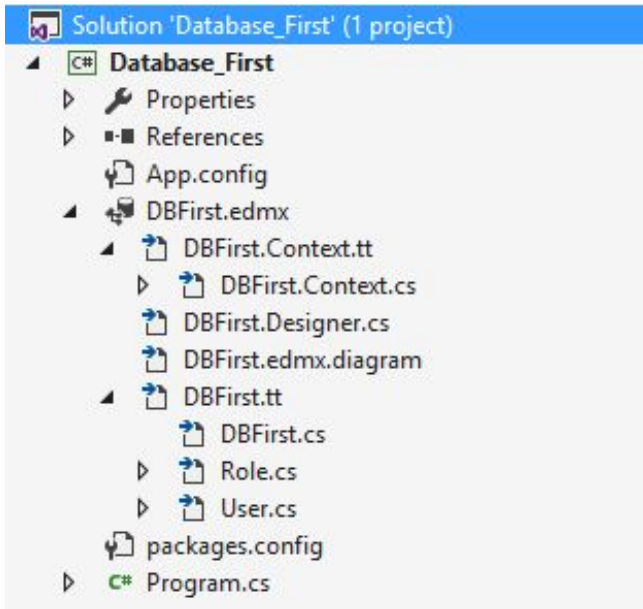
Database First : Connexion à la base de données



La base de données que l'on a sélectionnée contient deux tables.
Il n'y a pas de vue ni de procédure stockées donc rien à sélectionner.

Cliquez enfin sur "Finish".

Database First : Connexion à la base de données



Et voici le résultat que l'on obtient.

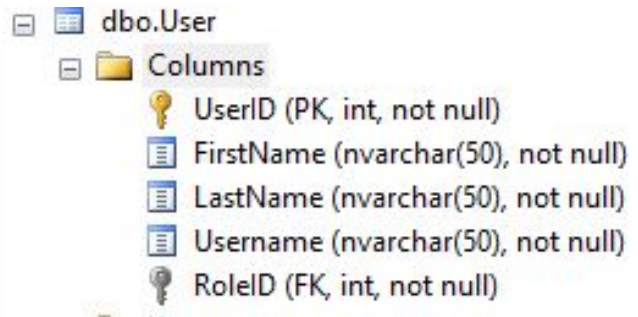
Entity Framework a généré un fichier edmx (Entity Data Model).

Ce fichier contient plusieurs fichiers, il y en a deux qui vont particulièrement nous intéresser.:

- DBFirst.context.tt qui contient le DbContext qui permettra l'accès à la DB par le développeur
- DBFirst.tt qui contient les modèles générés sur base de la base de données elle-même.

Database First : Modification de la structure

Notre table User est représentée comme ceci :

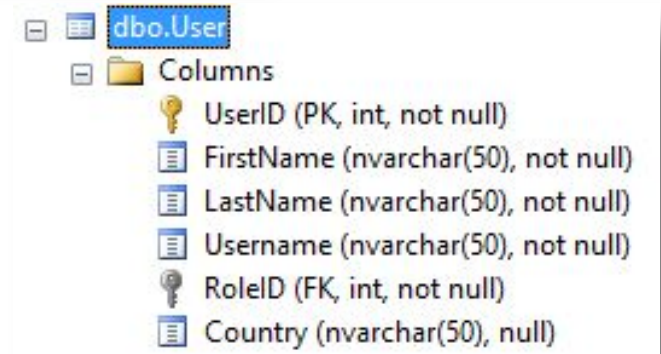


Nous ajoutons maintenant un champs "Country" à notre base de données :

```
USE [Database_First]
GO

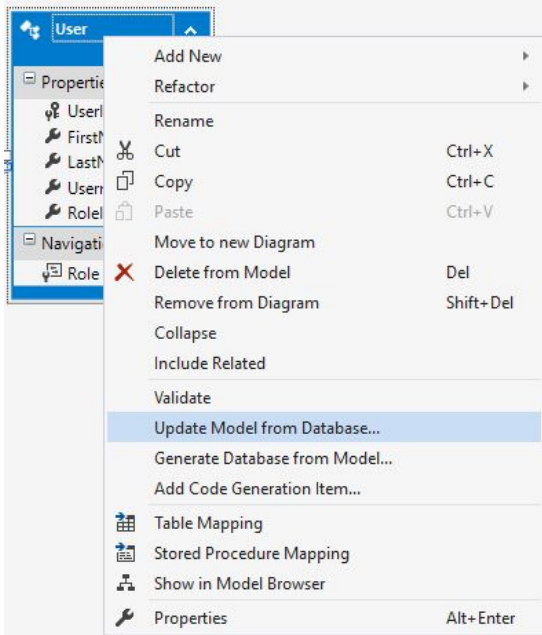
ALTER TABLE [User]
    ADD [Country] NVARCHAR(50);
GO
```

Voici le résultat :



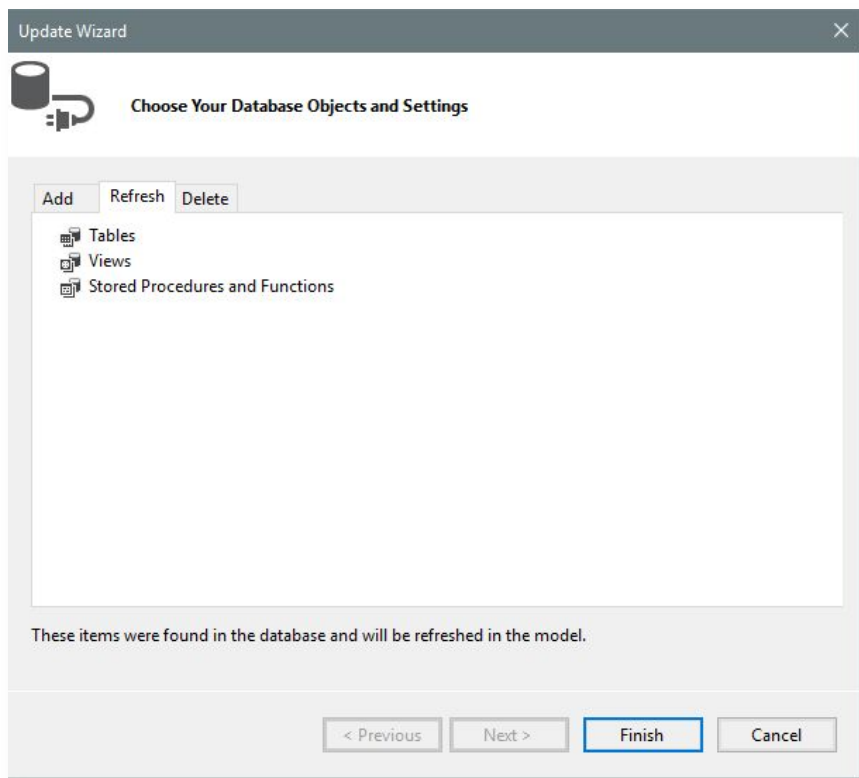
Database First : Modification de la structure

Notre modèle User ne correspond donc plus à notre base de données. Pour appliquer ces changements, il suffit d'ouvrir le diagramme de classe en double cliquant sur le fichier DBFirst.edmx. Ensuite, clic droit sur l'entité "User"



Sélectionnez "Update Model from Database".

Database First : Modification de la structure



Sélectionnez l'onglet "Refresh" et cliquez sur "Finish".

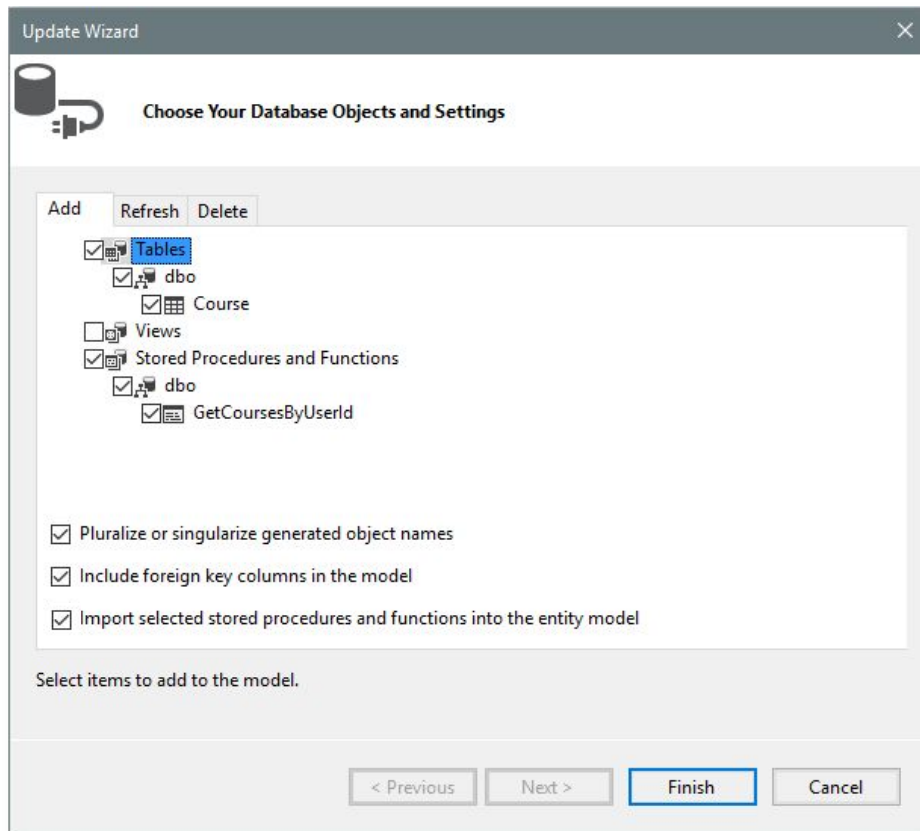
Database First : Modification de la structure

On peut voir que notre modèle à bien changé.

```
public partial class User
{
    public int UserID { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Username { get; set; }
    public int RoleID { get; set; }
    public string Country { get; set; }

    public virtual Role Role { get; set; }
}
```

Database First : Procédure stockée



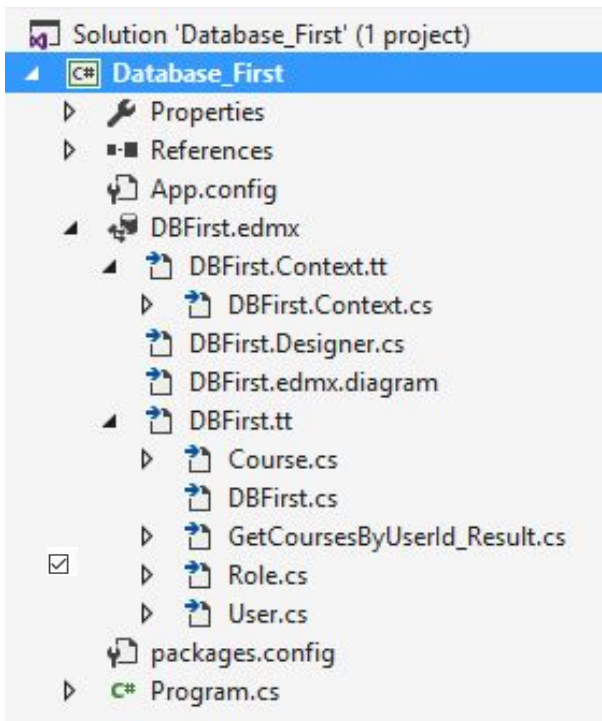
On peut également importer les procédures stockées depuis le serveur SQL. Pour cela, exécutez le fichier “Database_First_Update2.sql” et “Database_First_Update3.sql”.

La procédure stockée va nous permettre de récupérer la liste des cours pour un utilisateur donné.

Refaites l'opération vue avant pour mettre à jour les modèles depuis la base de données.

Cette fois, restez sur l'onglet “Add” et sélectionnez la nouvelle table “Course” et la procédure stockée puis cliquez sur “Finish”.

Database First : Procédure stockée



On peut voir qu'un modèle "GetCoursesByUserId_Result.cs" a été ajouté à notre solution.

Cette classe est le modèle que la procédure stockée va utiliser pour nous renvoyer le résultat.

Et qu'une nouvelle méthode a été ajoutée à notre DBFirstContext

```
public virtual ObjectResult<GetCoursesByUserId_Result>  
    GetCoursesByUserId(Nullable<int> userID)  
{  
    var userIDParameter = userID.HasValue ?  
        new ObjectParameter("UserID", userID) :  
        new ObjectParameter("UserID", typeof(int));  
  
    return ((IObjectContextAdapter)  
        this).ObjectContext.ExecuteFunction<GetCoursesByUserId_Result>  
        ("GetCoursesByUserId", userIDParameter);  
}
```

Database First : Procédure stockée

On peut maintenant vérifier que la procédure stockée fonctionne :

```
using (DBFirstContext db = new DBFirstContext())
{
    User _aurelien = db.Users.FirstOrDefault(u => u.FirstName == "Aurélien");

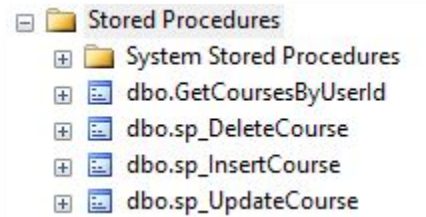
    foreach (GetCoursesById_Result course in db.GetCoursesById(_aurelien.UserID))
    {
        Console.WriteLine(course.Name);
    }
    Console.ReadKey();
}
```



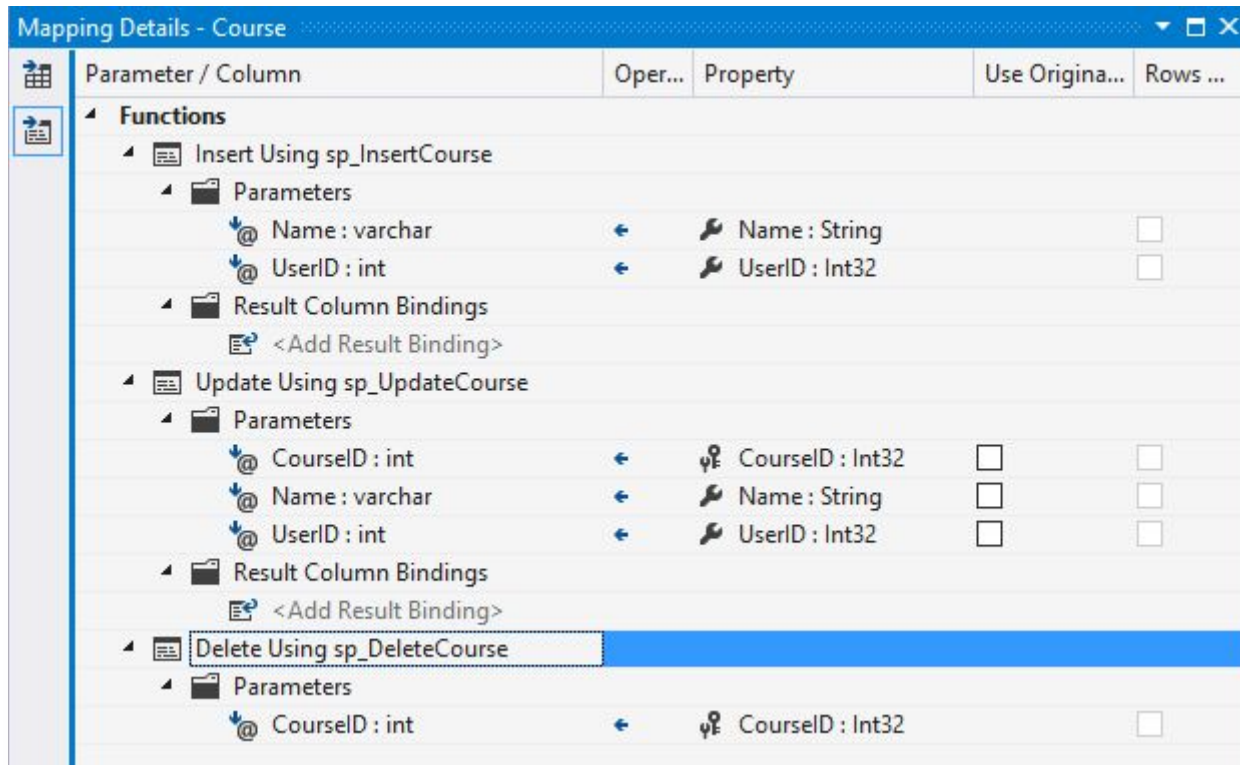
Database First : Procédure stockée

On peut même aller un peu plus loin avec les procédures stockées.

Exécutez les fichiers “Database_First_Update4.sql”, “Database_First_Update5.sql” et “Database_First_Update6.sql”. Cela va créer 3 procédures stockées sur la table “Course”.



Database First : CRUD avec Procédures Stockées



Importez les procédures stockées comme on l'a déjà vu.

On va maintenant mapper ces procédures avec l'entité "Course".

Dans la fenêtre "Mapping Details", cliquez sur <Select insert function> et ajouter les 3 valeurs (insert, update, delete).

La propriété est automatiquement sélectionnée, vous pouvez la modifier si nécessaire.

N'oubliez pas d'enregistrer votre diagramme pour valider les modifications.

Database First : Exemple d'utilisation

Voici un exemple d'utilisation. Il n'y rien de plus à faire qu'habituellement, la procédure stockée est appelée automatiquement.

```
using (DBFirstContext db = new DBFirstContext())
{
    Course _asp = new Course()
    {
        Name = "asp .net",
        UserID = db.Users.FirstOrDefault(u => u.FirstName == "Aurélien").UserID
    };

    //lancement de la procédure stockée sp_InsertCourse
    db.Courses.Add(_asp);
    db.SaveChanges();

    Console.WriteLine("Course listing.");
    foreach (Course _course in db.Courses)
    {
        Console.WriteLine("\t" + _course.Name);
    }
    Console.ReadKey();
    Console.Clear();|
}
```

```
//lancement de la procédure stockée sp_UpdateCourse
Course _newAsp = db.Courses.FirstOrDefault(c => c.Name == "asp .net");
_newAsp.Name = _newAsp.Name.ToUpper();
db.SaveChanges();

Console.WriteLine("New Name");
Console.WriteLine(_newAsp.Name);
Console.WriteLine("-----");

//lancement de la procédure stockée sp_DeleteCourse
db.Courses.Remove(_newAsp);
db.SaveChanges();

Console.WriteLine("");
Console.WriteLine("Course listing.");
foreach (Course _course in db.Courses)
{
    Console.WriteLine("\t" + _course.Name);
}
```

Code First to New Database

Le but de Code-first to New Database est de créer nos modèles tel que nous l'entendons, on ne se préoccupe donc pas de la base de données, elle sera créée automatiquement par Entity Framework lors de la première exécution du projet.

Voici les étapes que nous allons remplir pour que notre projet puisse aboutir :

- Créer la connectionString : nous allons indiquer à Entity Framework le type de base de données et les informations de connexion à cette base de données.
- Créer les modèles (classes) que nous utiliserons dans notre applications
- Créer le DbContext

Code First : connectionString

Afin qu'Entity Framework puisse se connecter à notre système de gestion de base de données (SQL Server dans ce cas), nous allons devoir modifier le fichier app.config (nous sommes en console) pour lui ajouter la connectionString. Dans un environnement ASP.NET, nous devrions éditer le fichier Web.config.

Comme vous pouvez le constater, le fichier app.config est un fichier XML auquel nous allons devoir ajouter un noeud.. Placez ce code juste avant la dernière fermeture de balise, en l'occurrence : "</configuration>."

```
<connectionStrings>  
  <add name="DemoCodeFirstContext" connectionString="data source=(local)\sql;initial catalog=Model.DemoCodeFirst;integrated  
security=True;MultipleActiveResultSets=True;App=EntityFramework" providerName="System.Data.SqlClient"/>  
</connectionStrings>
```

Changez le "name" par le nom que vous désirez, par convention, on indique le nom du projet suivi du mot "Context".

Data source doit également correspondre à votre serveur SQL. Dans un environnement de développement, on peut également utiliser le localdb de Visual Studio en remplaçant le data source par : "(LocalDb)\MSSQLLocalDB"

Integrated security à True indique que vous n'utilisez pas de login et mot de passe pour la connexion à la DB. C'est le login/mot de passe qui est utilisé à la place.

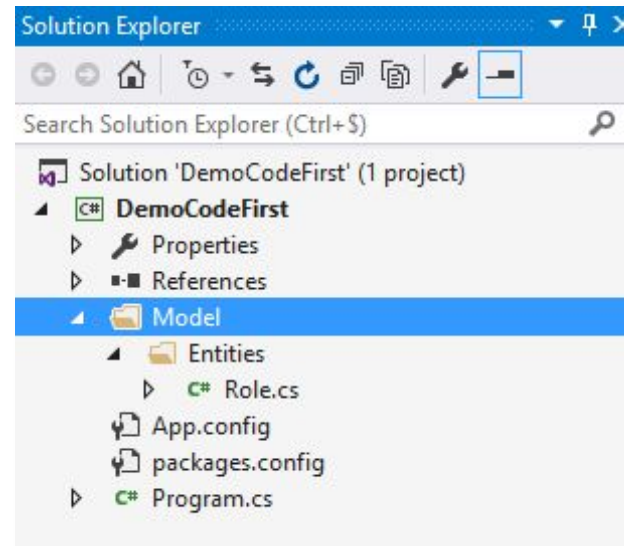
Code First : Création des Modèles

Afin qu'Entity Framework puisse créer la base de données, nous avons besoin de lui définir les objets à utiliser pour la création des tables et des champs de ces tables.

Les modèles que nous allons créer sont de simples objets (classes) appelés POCO (Plain Old CLR Object).

Voici une classe "Role" que nous allons utiliser pour l'exemple (vu que d'autres classes vont venir s'ajouter , je vous conseille de créer un répertoire "Model" ainsi qu'un sous répertoire "Entities" pour une meilleure lisibilité) :

```
class Role
{
    public int RoleID { get; set; }
    public string Name { get; set; }
}
```



Et voici l'état actuel de notre explorateur de solution.

Code First : Création du DbContext

```
using DemoCodeFirst.Model.Entities;
using System;
using System.Collections.Generic;
using System.Data.Entity;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace DemoCodeFirst.Model
{
    class DemoCodeFirstContext : DbContext
    {
        public virtual DbSet<Role> Roles { get; set; }

        public DemoCodeFirstContext()
            :base("name=DemoCodeFirstContext")
        { }
    }
}
```

Maintenant que notre première classe a été créée, nous pouvons créer le DbContext.

A la racine du répertoire "Model", créez une nouvelle classe que vous appelez "DemoCodeFirstContext" :

Cette classe hérite de la classe DbContext, nous avons donc besoin d'ajouter un namespace à notre classe.

```
using System.Data.Entity;
```

Nous avons également besoin du using pour pouvoir accéder à notre modèle "Role".

```
using DemoCodeFirst.Model.Entities;
```

DbSet<TEntity> représente la collection de toutes les entités de TEntity.

Code First : Création du DbContext

```
public DemoCodeFirstContext()  
    :base("name=DemoCodeFirstContext")  
{ }
```

Nous avons également ajouté un constructeur à notre classe qui fait appel à la classe de base, DbContext en l'occurrence.

```
//  
// Summary:  
//     Constructs a new context instance using the given string as the name or connection  
//     string for the database to which a connection will be made. See the class remarks  
//     for how this is used to create a connection.  
//  
// Parameters:  
//     nameOrConnectionString:  
//         Either the database name or a connection string.  
[SuppressMessage("Microsoft.Reliability", "CA2000:Dispose objects before losing scope")]  
[SuppressMessage("Microsoft.Usage", "CA2214:DoNotCallOverridableMethodsInConstructors")]  
public DbContext(string nameOrConnectionString);
```

Si on jette un oeil sur la classe DbContext, on s'aperçoit que le premier constructeur de cette classe demande soit le nom de la connectionString, soit la connectionString elle même.

Vu que nous avons déjà défini notre connectionString dans le fichier app.config, il nous suffit de lui passer le nom en paramètre.

Code First : Test

A ce stade, si nous lançons le projet, vous constaterez qu'aucune base de données n'a été créée sur notre serveur SQL. En effet, nous ne demandons rien à notre application. Entity Framework n'ayant pas besoin d'accéder à la DB, il ne fait aucune requête et ne crée donc pas cette DB.

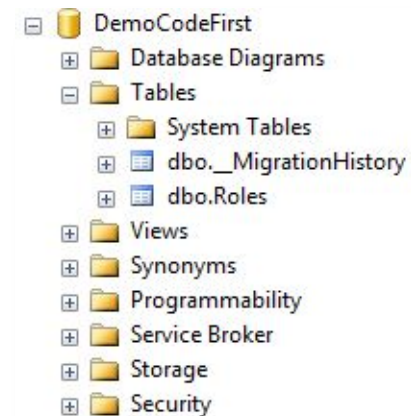
Ajoutons un rôle puis exécutons notre programme.

```
using(DemoCodeFirstContext db = new DemoCodeFirstContext())
{
    Role _role = new Role()
    {
        Name = "Admin"
    };

    db.Roles.Add(_role);
    db.SaveChanges();

    Console.WriteLine("L'ajout du rôle s'est correctement déroulé");
    Console.ReadKey();
}
```

La db a bien été créée et la table rôle contient bien mon nouveau rôle "Admin".



	RoleID	Name
1	1	Admin

Code First : Les relations entre les tables.

Le but d'une base de données relationnelle, c'est justement de pouvoir créer des relations entre les tables.

Entity Framework en approche Code-First permet bien évidemment de créer des relations.

Les relations possibles sont au nombre de 3 :

- One to Many
- One to One
- Many to Many

Créons une nouvelle classe "User" comme ceci :

```
class User
{
    public int UserID { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Username { get; set; }
    public bool IsActive { get; set; }
}
```

Code First : relation One to Many

A ce stade-ci, nous avons deux tables totalement séparées l'une de l'autre.

Pour créer la relation entre "User" et "Role", nous allons simplement ajouter une propriété virtuelle à la classe "User" ayant le type "Role".

Entity Framework va comprendre qu'il doit créer la relation entre les deux tables grâce au mot clé "virtual". N'oublions pas non plus d'ajouter notre DbSet<User> au contexte. La classe "Role" va quand à elle contenir la liste des "User".

```
class User
{
    public int UserID { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Username { get; set; }
    public bool IsActive { get; set; }

    public virtual Role Role { get; set; }
}
```

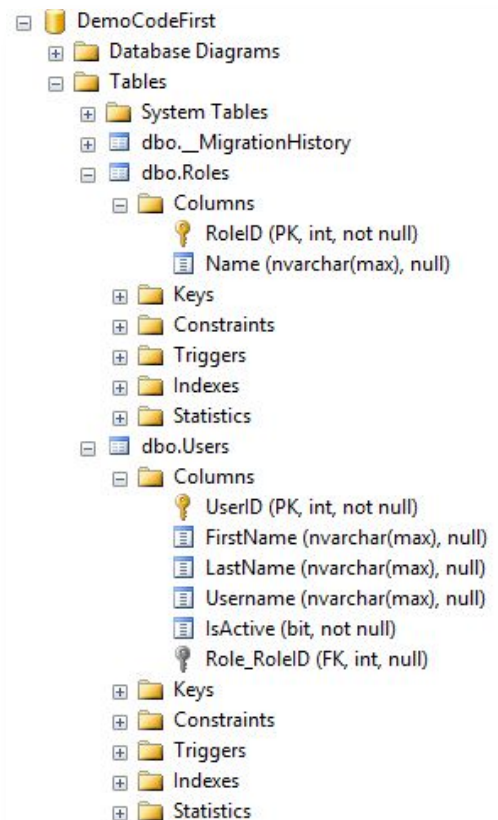
```
class DemoCodeFirstContext : DbContext
{
    public virtual DbSet<Role> Roles { get; set; }
    public virtual DbSet<User> Users { get; set; }

    public DemoCodeFirstContext()
        :base("name=DemoCodeFirstContext")
    { }
}
```

Code First : relation One to Many

Pour tester, supprimez d'abord votre base de données actuelle, nous verrons plus loin comment faire pour mettre à jour la base de données pour ne pas devoir systématiquement la supprimer

On constate que la structure a changé et que la table "User" a bien été ajoutée.



Code First : relation One to One

Ajoutons une nouvelle classe à nos entités : “Address”. Nous allons créer une relation One To One avec la table “User” (On part du principe qu’un utilisateur a obligatoirement une et une seule adresse).

```
class Address
{
    public int AddressID { get; set; }
    public string Street { get; set; }
    public string City { get; set; }
    public int PostalCode { get; set; }
    public string Country { get; set; }

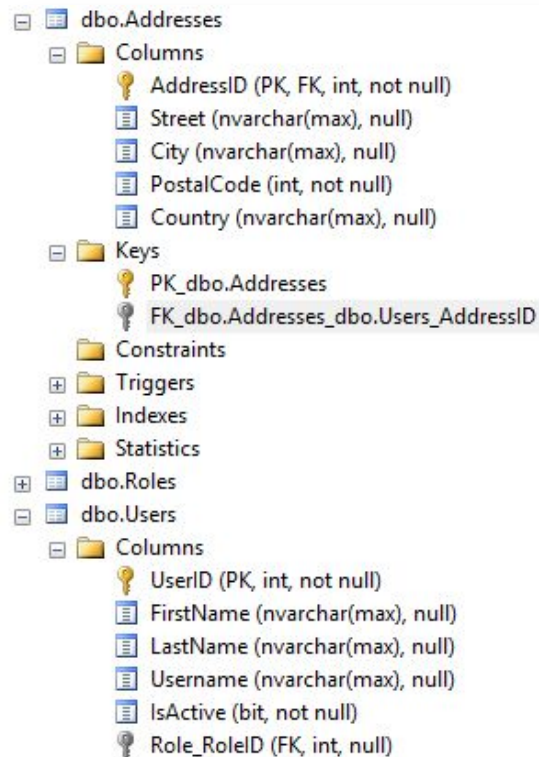
    public virtual User User { get; set; }
}
```

Pour finaliser la relation, ajoutons la propriété Address à notre classe User.

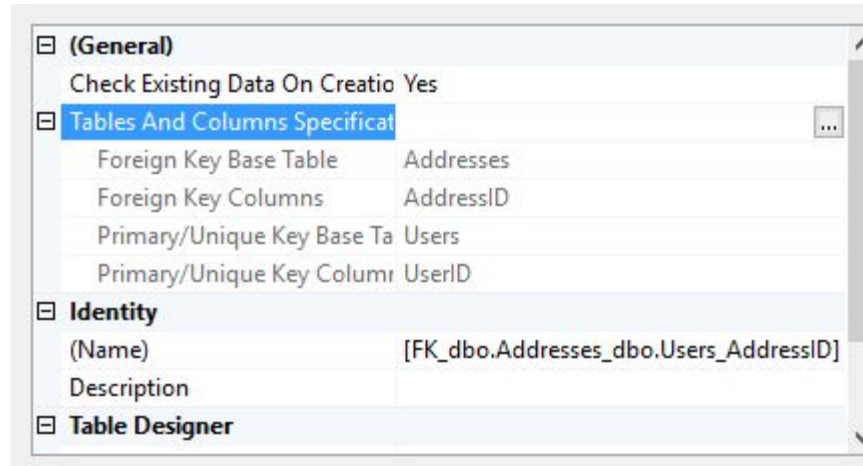
```
class User
{
    public int UserID { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Username { get; set; }
    public bool IsActive { get; set; }

    public virtual Role Role { get; set; }
    public virtual Address Address { get; set; }
}
```

Code First : relation One to One



On peut voir à gauche que la relation entre les deux tables a bien été créée.
Voici un peu plus en détail :



Code First : relation Many to Many

Créons maintenant une nouvelle classe "Course". Elle va nous servir à créer une relation avec "User".

```
class Course
{
    public int CourseID { get; set; }
    public string Name { get; set; }

    public virtual ICollection<User> Users { get; set; }

    public Course()
    {
        this.Users = new HashSet<User>();
    }
}
```

Notre objet "Course" contiendra donc une liste de Users.

Remarquez également que l'on a ajouté un constructeur à cette classe pour instancier la liste des Users de cette même classe.

Code First : relation Many to Many

Maintenant, notre classe "User".

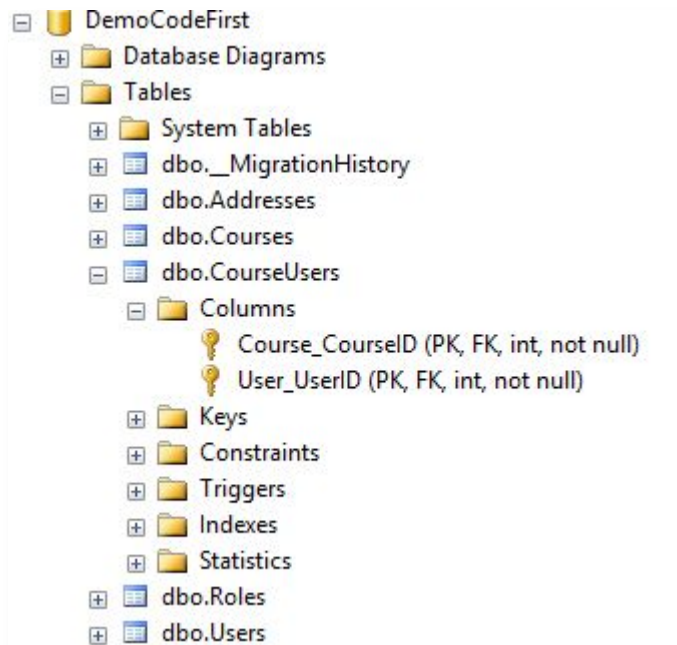
On ajoute également une liste de "Course" et on instancie cette ICollection dans le constructeur, comme précédemment.

```
class User
{
    public int UserID { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Username { get; set; }
    public bool IsActive { get; set; }

    public virtual Role Role { get; set; }
    public virtual Address Address { get; set; }
    public virtual ICollection<Course> Courses { get; set; }

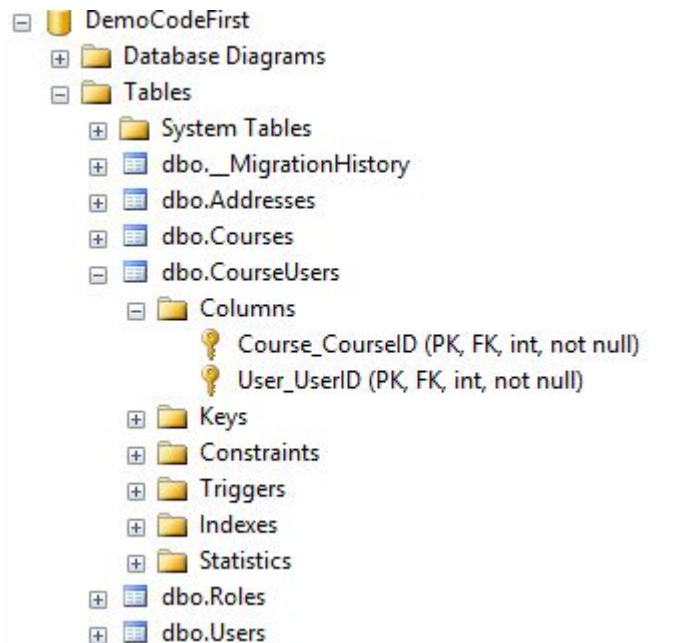
    public User()
    {
        this.Courses = new HashSet<Course>();
    }
}
```

Code First : relation Many to Many



On voit ici le résultat escompté. Entity Framework a créé une table intermédiaire composée du nom de la table Courses et Users.

Code First : relation Many to Many



On voit ici le résultat escompté. Entity Framework a créé une table intermédiaire composée du nom de la table Courses et Users.

Code First : Data Annotation

Pour le moment, vous aurez sans doute remarqué que tout cela n'est pas très précis.

Nous n'avons pas encore pu définir qu'un champ est unique, non null, etc...

Pour ce faire, il existe les "Data Annotation".

Nous avons déjà utilisé une Data Annotation dans la classe "Address" :

```
[ForeignKey("User")]  
public int AddressID { get; set; }
```

Il en existe un certain nombre que nous allons détailler.

Une annotation se place systématiquement entre [], certaines peuvent offrir différentes options à l'aide d'une parenthèse, comme ci-dessus.

Deux espaces de noms sont nécessaires suivants les annotations utilisées.:

- System.ComponentModel.DataAnnotations;
- System.ComponentModel.DataAnnotations.Schema;

Code First : Data Annotation

[Table]

```
using System.ComponentModel.DataAnnotations.Schema;

[Table("Utilisateur")]
class User
{
    public int UserID { get; set; }
    public string Name { get; set; }
}
```

La table dans la base de données sera nommée “Utilisateur” au lieu de “User”.

Code First : Data Annotation

[Column]

```
using System.ComponentModel.DataAnnotations.Schema;

class User
{
    public int UserID { get; set; }
    [Column("Nom")]
    public string Name { get; set; }
}
```

Même principe que pour l'annotation [Table], sauf que cela concerne la colonne.

Code First : Data Annotation

[Key]

Défini la clé primaire de la table.
Cette annotation peut être placée sur deux champs afin de créer une clé composée.

```
using System.ComponentModel.DataAnnotations;

class User
{
    [Key]
    public int UserID { get; set; }
    public string Name { get; set; }
}
```

Code First : Data Annotation

[NotMapped]

```
using System.ComponentModel.DataAnnotations.Schema;

class User
{
    public int UserID { get; set; }
    public string Name { get; set; }
    public string FirstName { get; set; }

    [NotMapped]
    public string Username
    {
        get
        {
            return this.FirstName.Substring(0, 1).ToUpper()
                + this.Name.ToLower();
        }
    }
}
```

Permet de ne pas ajouter une propriété à la base de données.
Dans le cas d'une propriété calculée par exemple...

Code First : Data Annotation

[ForeignKey]

```
using DemoCodeFirst.Model.Entities;
using System.ComponentModel.DataAnnotations.Schema;

class User
{
    [ForeignKey("Address")]
    public int UserID { get; set; }
    public string Name { get; set; }

    public virtual Address Address { get; set; }
}
```

Permet de définir une clé étrangère, comme nous l'avons fait pour la classe "Address". Le paramètre string doit obligatoirement représenter la propriété à laquelle vous voulez la lier.

Code First : Data Annotation

[Index]

```
using DemoCodeFirst.Model.Entities;
using System.ComponentModel.DataAnnotations.Schema;

class User
{
    [ForeignKey("Address")]
    public int UserID { get; set; }
    public string Name { get; set; }

    public virtual Address Address { get; set; }
}
```

Permet de définir l'index de la table. Il y a trois surcharges disponibles sur l'index.

Code First : Data Annotation

[Required]

```
using System.ComponentModel.DataAnnotations;

class User
{
    public int UserID { get; set; }
    [Required]
    public string Name { get; set; }
}
```

Indique qu'une propriété est obligatoire.
Le champs de la table sera donc indiqué comme "Not Null".

Code First : Data Annotation

[MaxLength]

```
using System.ComponentModel.DataAnnotations;

class User
{
    public int UserID { get; set; }
    [Required]
    public string Name { get; set; }
}
```

Permet de définir la longueur maximum d'un champs.
si ce n'est pas spécifié, la longueur maximum est prise par défaut.

Code First : Data Annotation

[MinLength]

Permet de définir la longueur minimum d'un champs.

```
using System.ComponentModel.DataAnnotations;

class User
{
    public int UserID { get; set; }
    [Required]
    public string Name { get; set; }
}
```

Code First : Héritage

L'héritage de table en SQL s'apparente tout à fait à l'héritage de classe dans un langage de programmation tel que C#, Java, etc.

Il existe trois types d'héritage que l'on peut implémenter à l'aide d'Entity Framework Code-First :

- **Table per Hierarchy (TPH)** : Cette approche réunit toutes les classes et toutes les propriétés de chaque classe dans une seule table. C'est l'approche par défaut d'Entity Framework.
- **Table per Type (TPT)** : Cette approche suggère une seule table par classe. Chaque classe sera représentée par une table dans la base de données, que la classe soit abstraite ou concrète.
- **Table per Concrete Type (TPC)** : Cette approche suggère une table par classe concrète, mais pas pour la classe abstraite. Donc, si vous héritez de la classe abstraite dans plusieurs classes concrètes, les propriétés de la classe abstraite feront partie de chaque table de chaque classe concrète.

Code First : Table per Hierachy

Considérons les deux classes suivantes :

```
class Professor
{
    public int ProfessorID { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Phone { get; set; }
    public string Email { get; set; }
    public DateTime HiringDate { get; set; }
}
```

```
class Student
{
    public int StudentID { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Phone { get; set; }
    public string Email { get; set; }
    public DateTime EnrollmentDate { get; set; }
}
```

On remarque très facilement que les deux classes possèdent des propriétés équivalentes.

Nous allons donc regrouper ces propriétés dans une classe abstraite "Person".

Code First : Table per Hierachy

Nous obtenons donc le résultat suivant :

```
abstract class Person
{
    public int PersonID { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Phone { get; set; }
    public string Email { get; set; }
}
```

```
class Professor : Person
{
    public DateTime HiringDate { get; set; }
}
```

```
class Student : Person
{
    public DateTime EnrollmentDate { get; set; }
}
```

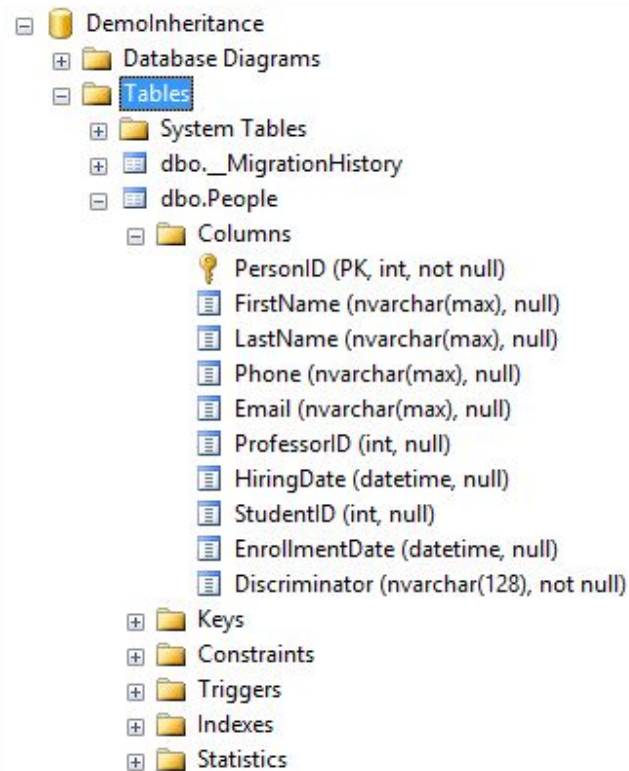
Code First : Table per Hierachy

Il nous reste maintenant à créer notre DbContext :

```
class DemoInheritanceContext : DbContext
{
    public virtual DbSet<Person> Persons { get; set; }
}
```

Le seul DbSet utile ici est la classe mère (abstraite).

Code First : Table per Hierachy



Et voici le résultat obtenu dans la base de données.

Vous remarquerez l'ajout d'un champs "discriminator". Ce champs est utilisé par Entity Framework afin de différencier un étudiant d'un professeur.

Voici le résultat dans la base de données après l'ajout d'un professeur et d'un étudiant.

PersonID	FirstName	LastName	Phone	Email	HiringDate	EnrollmentDate	Discriminator
1	Mike	Person	+32475000000	michael.person@btsom.be	2009-01-01 00:00:00.000	NULL	Professor
2	John	Doe	+32475111111	john.doe@gmail.com	NULL	2017-01-01 00:00:00.000	Student

Code First : Table per Type

Pour cette approche, nous avons les mêmes classes que précédemment, mis à part le fait que l'on a ajouté une annotation [Table] au dessus des classes concrètes : C'est ce qui permet à Entity Framework de comprendre qu'il doit créer 3 tables (1 classe abstraite, 2 classes concrètes).

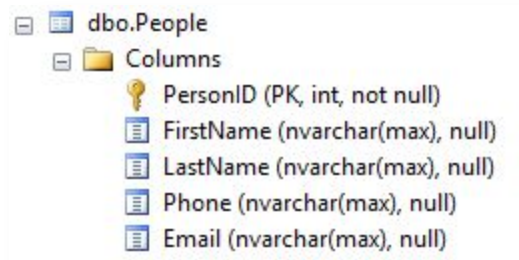
```
abstract class Person
{
    public int PersonID { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Phone { get; set; }
    public string Email { get; set; }
}
```

```
[Table("Professor")]
class Professor : Person
{
    public DateTime HiringDate { get; set; }
}
```

```
[Table("Student")]
class Student : Person
{
    public DateTime EnrollmentDate { get; set; }
}
```

Code First : Table per Type

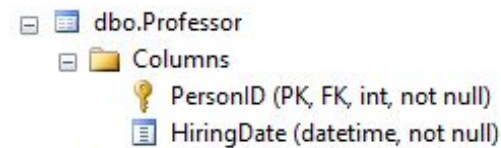
Et voici le résultat en base de données



dbo.People

- Columns
 - PersonID (PK, int, not null)
 - FirstName (nvarchar(max), null)
 - LastName (nvarchar(max), null)
 - Phone (nvarchar(max), null)
 - Email (nvarchar(max), null)

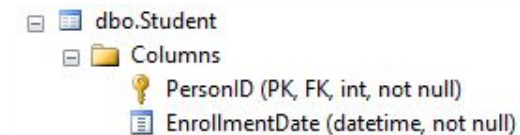
PersonID	FirstName	LastName	Phone	Email
1	Mike	Person	+32475000000	michael.person@btsom.be
2	John	Doe	+32475111111	john.doe@gmail.com



dbo.Professor

- Columns
 - PersonID (PK, FK, int, not null)
 - HiringDate (datetime, not null)

PersonID	HiringDate
1	2009-01-01 00:00:00.000



dbo.Student

- Columns
 - PersonID (PK, FK, int, not null)
 - EnrollmentDate (datetime, not null)

PersonID	EnrollmentDate
2	2017-01-01 00:00:00.000

On remarque que le clé primaire de la table People se retrouve comme clé étrangère dans les autres tables.

Code First : Table per Concrete Class

Dans ce cas de figure, c'est la classe abstraite qui prend un ID (le nom de cet id, nous le verrons, est utilisé dans les deux tables concrètes.

```
abstract class Person
{
    public int PersonID { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Phone { get; set; }
    public string Email { get; set; }
}
```

```
class Professor : Person
{
    public DateTime HiringDate { get; set; }
}
```

```
class Student : Person
{
    public DateTime EnrollmentDate { get; set; }
}
```

Code First : Table per Concrete Class

```
class DemoInheritanceContext : DbContext
{
    public virtual DbSet<Person> Persons { get; set; }

    protected override void OnModelCreating(DbModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Professor>().Map(m =>
        {
            m.MapInheritedProperties();
            m.ToTable("Professor");
        });

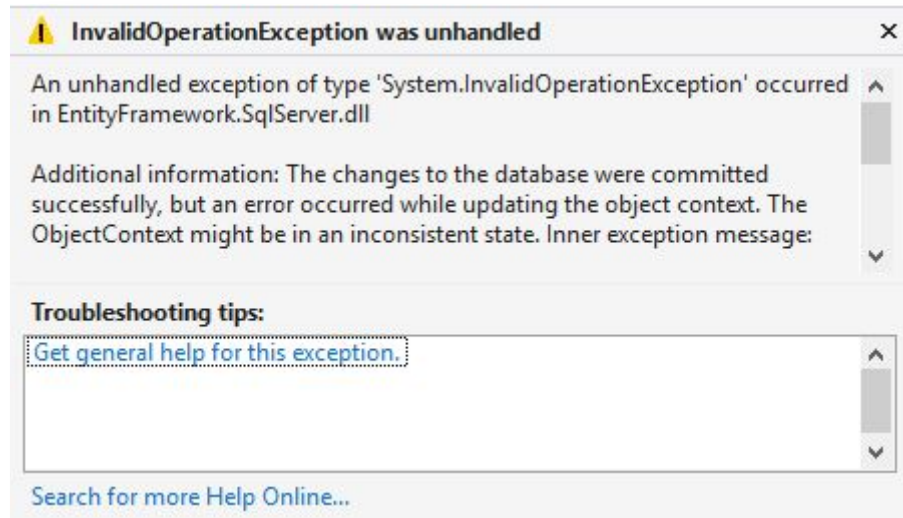
        modelBuilder.Entity<Student>().Map(m =>
        {
            m.MapInheritedProperties();
            m.ToTable("Student");
        });
    }
}
```

Et nous allons devoir réécrire la méthode
“OnModelCreating” venant de la classe DbContext;

Code First : Table per Concrete Class

Si on teste l'application, on s'aperçoit de deux choses :

L'application renvoie une exception



Mais les données ont bien été ajoutées à la base de données.

PersonID	FirstName	LastName	Phone	Email	HiringDate
0	Mike	Person	+32475000000	michael.person@btsom.be	2009-01-01 00:00:00.000

PersonID	FirstName	LastName	Phone	Email	EnrollmentDate
0	John	Doe	+32475111111	john.doe@gmail.com	2017-01-01 00:00:00.000

Code First : Table per Concrete Class

Cette exception est due au fait que Entity Framework tente d'accéder à des valeurs qui se trouvent dans la classe abstraite.

Les deux données qu'il trouve possède le même ID. La colonne de la table étant spécifiée comme clé primaire, la même valeur comme ID pose problème dans ce cas de figure.

Pour contourner ce problème, nous sommes obligés de générer nous même un ID unique (GUID) pour chaque objet que l'on crée. Nos classes auront donc cette forme

```
abstract class Person
{
    public Guid PersonID { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Phone { get; set; }
    public string Email { get; set; }
}

class Professor : Person
{
    public DateTime HiringDate { get; set; }
}

class Student : Person
{
    public DateTime EnrollmentDate { get; set; }
}
```

Code First : Table per Concrete Class

Notre programme se présente comme ceci :

```
using (DemoInheritanceContext db = new DemoInheritanceContext())
{
    Professor _prof = new Professor()
    {
        PersonID = Guid.NewGuid(),
        FirstName = "Mike",
        LastName = "Person",
        Email = "michael.person@btsorm.be",
        Phone = "+32475000000",
        HiringDate = new DateTime(2009,01,01)
    };
    db.Persons.Add(_prof);

    Student _student = new Student()
    {
        PersonID = Guid.NewGuid(),
        FirstName = "John",
        LastName = "Doe",
        Email = "john.doe@gmail.com",
        Phone = "+32475111111",
        EnrollmentDate = new DateTime(2017,01,01)
    };

    db.Persons.Add(_student);
    db.SaveChanges();
}
Console.ReadKey();
```

Code First : Table per Concrete Class

L'application n'a pas retourné d'exception et notre base de données contient bien nos deux enregistrement avec un ID unique

PersonID	FirstName	LastName	Phone	Email	HiringDate
64CFA54E-7218-4893-9838-0ACD60E0FBF5	Mike	Person	+32475000000	michael.person@btsom.be	2009-01-01 00:00:00.000

PersonID	FirstName	LastName	Phone	Email	EnrollmentDate
85CF286B-6C32-4ABE-8B46-9892DFDCB2C0	John	Doe	+32475111111	john.doe@gmail.com	2017-01-01 00:00:00.000

Code First : Migration

Avec tout ce que l'on a déjà vu, on peut déjà bien avancer mais je suppose que vous aurez remarqué que supprimer notre base de données manuellement et systématiquement n'est pas des plus pratiques.

Lorsqu'on met à jour un modèle ou que l'on ajoute un modèle, on peut procéder à une migration pour mettre à jour notre base de données.

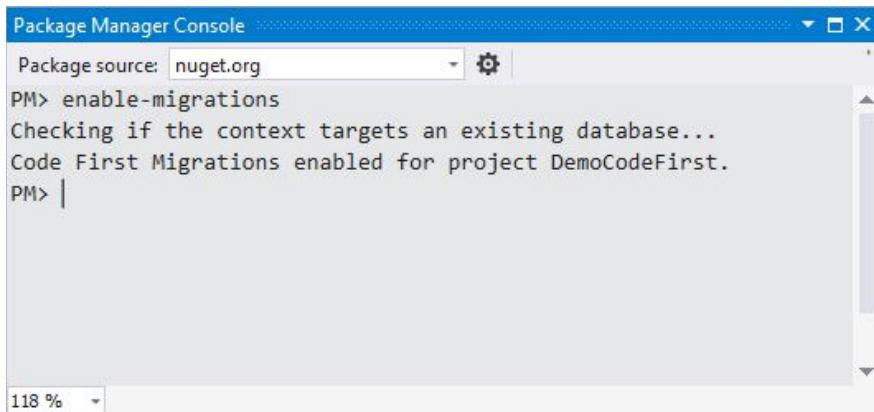
Code First : Migration

Avant de continuer, ouvrez le projet “DemoCodeFirst” et supprimez la base de données.

Il existe deux méthodes pour faire une migration :

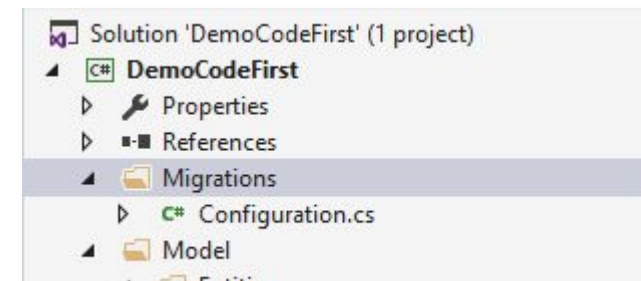
- Migration basée sur le code
- Migration automatisée

Pour le premier cas, il nous faut activer la migration et ouvrez Tools -> Nuget-Package Manager -> Package Manager Console. Dans la console, tapez la commande : enable-migrations



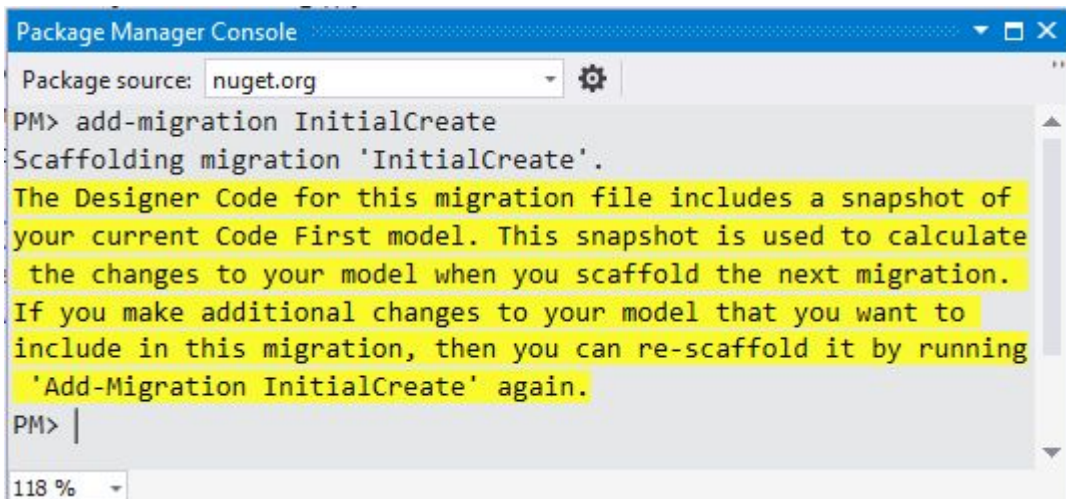
```
Package Manager Console
Package source: nuget.org
PM> enable-migrations
Checking if the context targets an existing database...
Code First Migrations enabled for project DemoCodeFirst.
PM> |
```

Cette commande crée un dossier “Migrations” et ajoute le fichier “Configuration.cs” que l’on peut éditer pour faire nos migrations automatisées.



Code First : Migration

Lorsque la migration a bien été activée, utilisez la commande `add-migration InitialCreate`



```
Package Manager Console
Package source: nuget.org
PM> add-migration InitialCreate
Scaffolding migration 'InitialCreate'.
The Designer Code for this migration file includes a snapshot of
your current Code First model. This snapshot is used to calculate
the changes to your model when you scaffold the next migration.
If you make additional changes to your model that you want to
include in this migration, then you can re-scaffold it by running
'Add-Migration InitialCreate' again.
PM> |
```

Un nouveau fichier contenant la date du jour et le terme “InitialCreate” va être créé dans le dossier Migrations.

Il contient 2 méthodes :

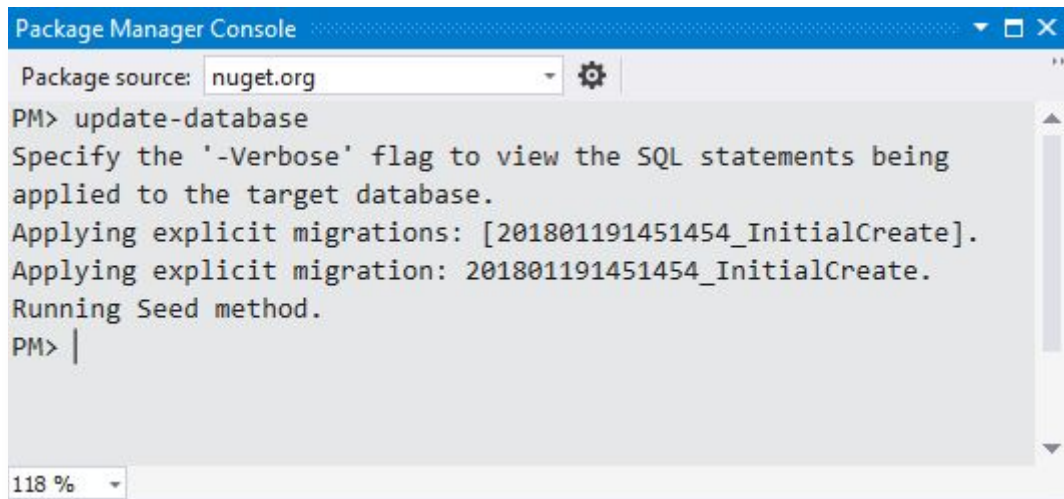
- `up()`
- `down()`

`up()` est utilisé lors d’une mise à jour

`down()` est utilisé lors d’un roll back.

Code First : Migration

Jusqu'à maintenant, nous avons préparé le terrain, si vous regardez dans votre SQL, vous constaterez qu'aucune base de données n'a été créée. Pour "valider" ces modifications, il nous suffit d'appliquer la commande : `update-database`



```
Package Manager Console
Package source: nuget.org
PM> update-database
Specify the '-Verbose' flag to view the SQL statements being
applied to the target database.
Applying explicit migrations: [201801191451454_InitialCreate].
Applying explicit migration: 201801191451454_InitialCreate.
Running Seed method.
PM> |
```

Suite à cette commande, la base de données a été créée suivant le méthode `up()` du fichier `{Date}_InitialCreate.cs`

Code First : Migration

Modifions maintenant la classe “User” comme ceci :

```
class User
{
    public int UserID { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public string Fullname { get; set; }
    public string Username { get; set; }
    public bool IsActive { get; set; }

    public virtual Role Role { get; set; }
    public virtual Address Address { get; set; }
    public virtual ICollection<Course> Courses { get; set; }

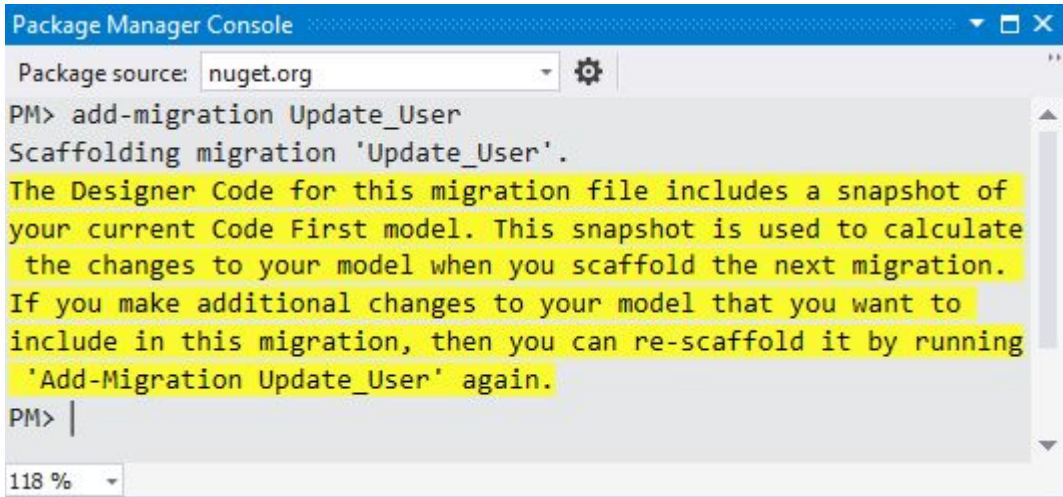
    public User()
    {
        this.Courses = new HashSet<Course>();
    }
}
```

La propriété Fullname a été ajouté.

Si vous lancez votre projet et essayez d'ajouter un nouvel utilisateur, vous allez avoir un joli message d'erreur qui vous indique que le contexte a changé depuis que la base de données a été créée.

Code First : Migration

Pour palier ce problème, nous allons exécuter notre première mise à jour de la base de donnée en utilisant les commandes suivantes :
“add-migrations Update_User”.



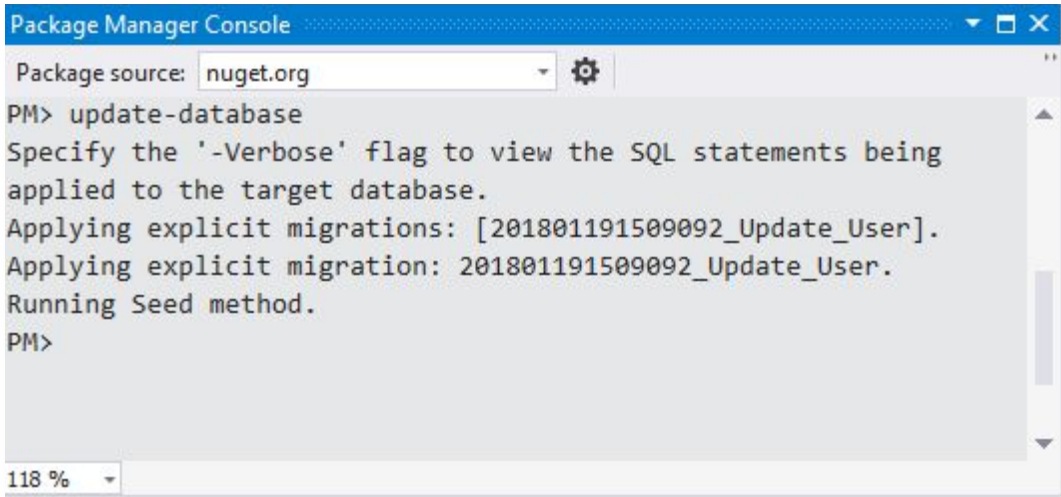
```
Package Manager Console
Package source: nuget.org
PM> add-migration Update_User
Scaffolding migration 'Update_User'.
The Designer Code for this migration file includes a snapshot of
your current Code First model. This snapshot is used to calculate
the changes to your model when you scaffold the next migration.
If you make additional changes to your model that you want to
include in this migration, then you can re-scaffold it by running
'Add-Migration Update_User' again.
PM> |
```



A la place de “Update_User”, vous pouvez choisir le nom qui vous convient. Soyez juste explicite dans ce nom. Il faut qu’il soit clair pour tout le monde.

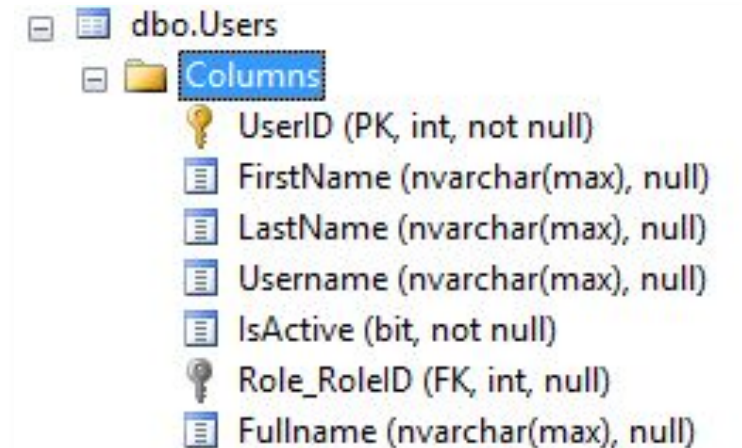
Code First : Migration

Pour valider les changements de la base de données, nous devons exécuter la commande “update-database”.



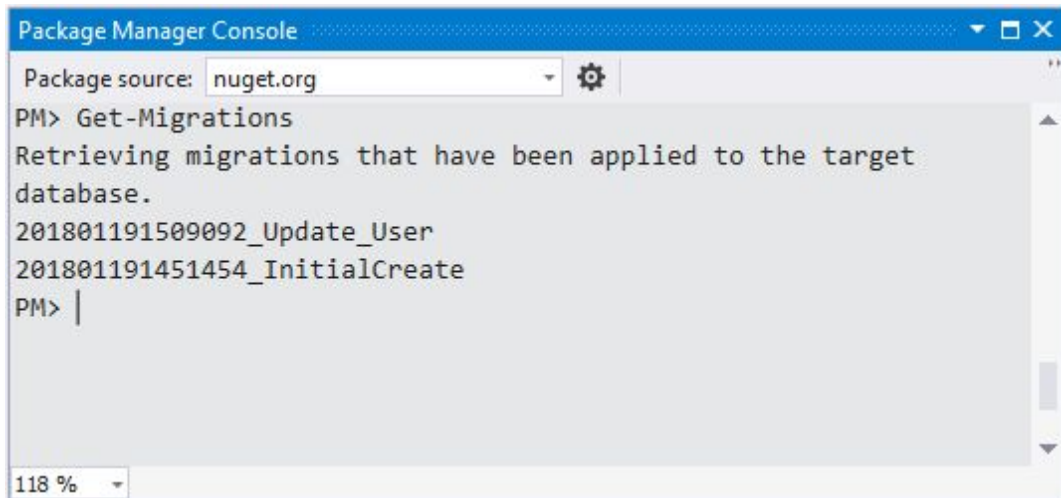
```
Package Manager Console
Package source: nuget.org
PM> update-database
Specify the '-Verbose' flag to view the SQL statements being
applied to the target database.
Applying explicit migrations: [201801191509092_Update_User].
Applying explicit migration: 201801191509092_Update_User.
Running Seed method.
PM>
```

La base de données à bien été mise à jour.



Code First : Migration

Si pour une raison quelconque vous devez revenir en arrière, vous pouvez le faire en suivant la procédure suivante.:

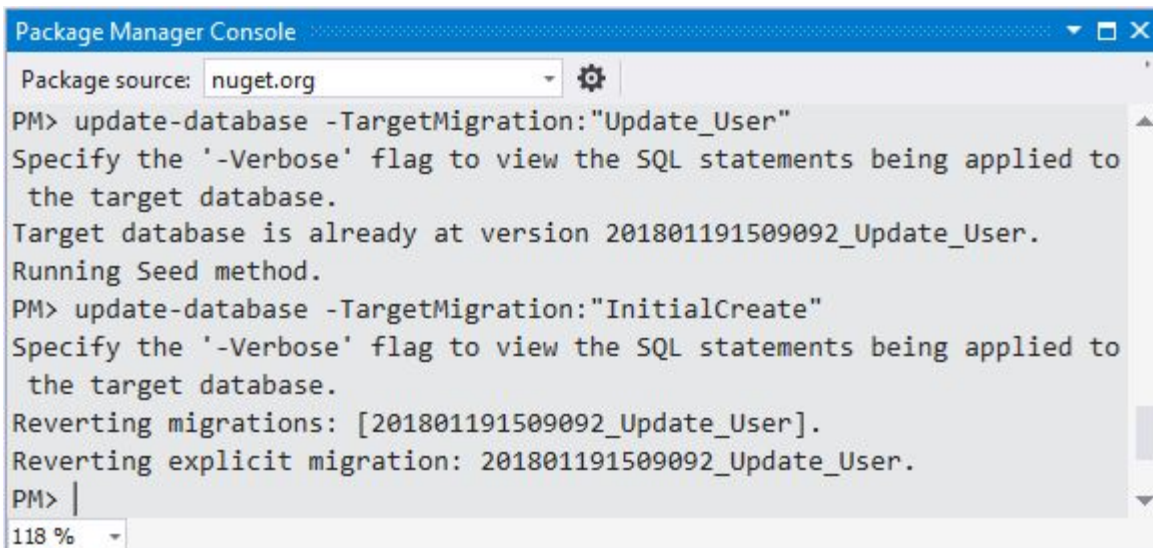


```
Package Manager Console
Package source: nuget.org
PM> Get-Migrations
Retrieving migrations that have been applied to the target
database.
201801191509092_Update_User
201801191451454_InitialCreate
PM> |
```

Get-Migrations : affiche une liste des migrations réussies

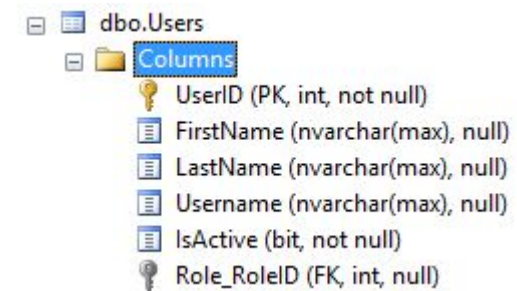
Code First : Migration

Pour faire le roll-back, utilisez la commande suivante : “update-database -TargetMigration:”InitialCreate” “. C’est le nom que vous avez choisi lors de l’ajout de la migration qui doit être utilisé pour le roll-back.



```
Package Manager Console
Package source: nuget.org
PM> update-database -TargetMigration:"Update_User"
Specify the '-Verbose' flag to view the SQL statements being applied to
the target database.
Target database is already at version 201801191509092_Update_User.
Running Seed method.
PM> update-database -TargetMigration:"InitialCreate"
Specify the '-Verbose' flag to view the SQL statements being applied to
the target database.
Reverting migrations: [201801191509092_Update_User].
Reverting explicit migration: 201801191509092_Update_User.
PM> |
118 %
```

La base de données est bien revenue à son état initial.



Si vous ne désirez pas que cette migration soit de nouveau exécutée lors de la prochaine mise à jour, n’oubliez pas de supprimer le fichier correspondant dans le répertoire “Migrations”